SPARK'S BASIC ARCHITECTURE

Spark is a distributed processing framework designed to handle large datasets across multiple machines (or nodes). Spark's key strength lies in its speed — it processes data up to 100 times faster than traditional systems like Hadoop MapReduce by storing intermediate…

Spark divides large datasets into smaller pieces and processes them simultaneously across multiple machines, ensuring fast, scalable data processing.

Apache Spark uses a **master-slave architecture** designed for fast, distributed data processing. At its core, a central "master" process coordinates tasks that are executed in parallel by "slave" processes across a cluster of machines.

Core Architecture Components

1. Spark Driver (The Master)

- **Function:** Acts as the "brain" of the application, managing the lifecycle and scheduling tasks.
- **Key Tasks:** It converts user code into a logical execution plan (DAG), monitors execution, and collects results from workers.
- **Spark Session:** The modern entry point (replacing the older `SparkContext`) for interacting with Spark's functionality.

2. Cluster Manager

- **Function:** An external service that allocates physical resources (CPU, memory) to Spark applications.

3. Executors (The Slaves)

- **Function:** Worker processes running on Worker Nodes that execute specific tasks assigned by the driver.
- **Data Management:** They store data in-memory or on disk for rapid access and report their status back to the driver.

4. Worker Nodes

- **Definition:** The physical or virtual machines in the cluster that host executor processes.

Cluster Manager acts as the primary "resource negotiator." It is an external service responsible for acquiring and managing physical resources—such as CPU and memory—across the cluster of machines to run Spark applications.

Key Responsibilities

- **Resource Allocation:** It decides which machines (nodes) are available and allocates resources (RAM and cores) to Spark applications based on their configuration.
- **Process Launching:** Once resources are allocated, the cluster manager launches **Executor** processes on the worker nodes on behalf of the Spark Driver.
- **Application Isolation:** It ensures that multiple Spark applications can run on the same cluster concurrently without interfering with each other's resources.
- **Monitoring and Health:** It monitors the health of the worker nodes and can manage failovers if a node or process fails.

SparkSession serves as the unified entry point for interacting with all Spark functionality. Introduced in Spark 2.0, it consolidates several legacy contexts—like `SparkContext`, `SQLContext`, and `HiveContext`—into a single interface, making it easier for developers to manage data without needing multiple entry points.

Key Roles of SparkSession in Architecture

- **Unified Entry Point:** It provides a single point of access to Spark features, including Spark SQL, DataFrame/Dataset APIs, and streaming.
- **Driver Program Coordinator:** The SparkSession resides in the Spark Driver, acting as the "brain" that translates user code into a logical plan for execution.
- **Encapsulation:** It internally manages the SparkContext, which is responsible for low-level resource management and communicating with the cluster manager.
- **Configuration Management:** It allows users to set runtime configurations and access the Catalog API to manage metadata like database and table names.

Spark Context serves as the primary entry point and the master coordinator for a Spark application. It resides within the **Driver Program** and acts as a gateway to the Spark cluster.

Core Role in Spark Architecture

- **Entry Point for Functionality:** It is the fundamental object required to access Spark's core features. Before Spark 2.0, it was the only way to interact with the cluster; in modern versions, it is usually managed through a SparkSession.
- **Connection to Cluster:** It establishes a connection between the user's application and the cluster, enabling the program to communicate with a Cluster Manager (like YARN, Mesos, or Kubernetes) to acquire resources like CPU and memory.
- **Resource Negotiation:** It communicates with the cluster manager to request worker nodes (Executors) for job execution.
- **Task Distribution:** It is responsible for breaking down a Spark job into smaller units called tasks and distributing them to the Executors on worker nodes.

Driver Node (or Driver Program) acts as the "brain" or central coordinator of the entire Spark application. It is the Java Virtual Machine (JVM) process that hosts the `main()` function and controls the execution flow of the application.

Core Responsibilities:

- **Maintains SparkContext/SparkSession:** It creates the SparkSession (or `SparkContext` in older versions), which serves as the primary entry point for the application to communicate with the Spark cluster.
- **Logical to Physical Planning:** It converts user-defined code into a Directed Acyclic Graph (DAG) of operations. The driver then optimizes this logical plan and transforms it into a physical execution plan consisting of stages and tasks.
- **Resource Negotiation:** It communicates with the **Cluster Manager** (e.g., YARN, Kubernetes, or Standalone) to request and negotiate the necessary physical resources (memory and CPU cores) to run executors on worker nodes.
- **Task Scheduling and Distribution:** Once resources are allocated, the driver's **Task Scheduler** distributes small units of work (tasks) to the Executors running on worker nodes.
- **Monitoring and Coordination:** It tracks the status of all running tasks, monitors executor health via heartbeats, and collects final results once tasks are complete.

Worker Node is a physical or virtual machine in the cluster that provides the necessary resources (CPU, RAM, and storage) to execute tasks. It acts as a slave node in Spark's master-slave model, receiving instructions from the Spark Driver to perform actual data processing.

Core Functions of a Worker Node

- **Resource Hosting**: Worker nodes provide the underlying hardware or virtualized environment where data processing occurs.
- **Task Execution**: They run specific units of work (tasks) sent by the Spark Driver.
- **Data Storage and Caching**: They store data partitions in memory (RAM) or on disk to speed up iterative operations and reuse data across multiple stages.
- **Inter-node Communication**: They handle the exchange of data between nodes, which is essential for operations like shuffles.

Client Mode is a deployment configuration where the Spark Driver runs on the machine where the job was submitted, rather than on a worker node within the cluster. This mode is primarily used for interactive development, debugging, and testing because it provides immediate feedback and console output.

Core Architecture in Client Mode

- **Driver Location:** The Spark Driver process is launched directly on the client machine (e.g., your laptop or a gateway/edge node).
- **Resource Management:** The driver communicates with the Cluster Manager (YARN, Standalone, or Kubernetes) to request resources for executors.
- **Application Master (AM):** In environments like YARN, an Application Master is still created within the cluster, but in client mode, it acts only as an executor launcher and resource negotiator, while the actual driver logic stays on the client.
- **Executor Communication:** Once launched on worker nodes, Executors communicate directly back to the driver on your client machine to receive tasks and return results.

Cluster Mode is a master-slave design specifically optimized for production environments. In this mode, the entire lifecycle of the application—from orchestration to execution—happens within the cluster, allowing the client machine to disconnect once the job is submitted.

Key Funstions of Cluster Mode

- **Fault Tolerance:** If the driver fails, the Cluster Manager can automatically restart it on a different node.
- **Resource Efficiency:** The driver runs close to the worker nodes (often on the same local network), reducing network latency for task scheduling.
- **Client Independence:** The client machine does not need to stay online for the duration of the job, making it ideal for long-running production tasks